

AI Resume Screening Platform

Final Project Report

Automated, Explainable, and Hybrid AI-Powered Candidate Screening

Abstract / Introduction

Hiring teams often receive far more resumes than they can carefully read. The AI Resume Screening Platform is a system built to solve this problem. It automates resume screening by reading bulk resumes in PDF and DOCX format, converting them into structured, machine-readable profiles, and ranking every candidate against a specific job description.

The platform uses a Hybrid Scoring approach. It combines an AI Semantic Score, which understands the meaning of a resume rather than just its keywords, with a Rule-Based Score, which checks exact skill overlap and years of experience. The result is a ranked shortlist of candidates, and for every candidate, the system generates a short, plain-language explanation of why they received that score. This report describes the problem the platform solves, how it is built, how it works internally, and the results it delivers.

Problem Statement

Recruiters are overwhelmed by the volume of resumes they receive for a single job opening. Screening hundreds of resumes by hand is slow and repetitive, and it takes recruiters away from higher-value work such as interviewing and hiring strategy.

This manual process causes two consistent problems:

- Manual fatigue — after reviewing many resumes in a row, reviewers lose focus and become less consistent, so similar candidates may be judged differently depending on when their resume was reviewed.
- Missed talent — strong candidates are sometimes skipped simply because a resume was skimmed too quickly, or because it used different wording than the job description. Traditional keyword-based systems make this worse, since they often reject a good match just because it uses a synonym instead of an exact keyword.

Proposed Solution

The **AI Resume Screening Platform** automates the initial hiring process. Recruiters upload a job description and multiple resumes, and the system:

1. Extracts key candidate information such as skills, experience, and education.
2. Matches candidates using AI-powered semantic analysis and rule-based checks.
3. Ranks candidates with clear explanations of their scores.

This reduces hours of manual screening to minutes while keeping recruiters in control of the final decision.

System Architecture

The platform is built on a 3-tier architecture. This means the system is divided into three independent layers, so each part can be built, scaled, and maintained on its own.

1. Presentation Tier — Streamlit Frontend

The interface where recruiters upload a job description and a batch of resumes, and where they view the final ranked dashboard of candidates.

2. Application Tier — FastAPI Backend

The engine of the platform. It handles all business logic: parsing resumes, calling the AI models, calculating hybrid scores, and communicating with the databases.

3. Data Tier — PostgreSQL, Qdrant, and Redis

Stores everything the system needs: structured candidate data, vector embeddings for semantic search, and cached data for speed and protection.

Resume parsing and AI scoring take time, since each resume must be read and analyzed by an AI model. To keep the platform responsive, these steps run as asynchronous background tasks. When a recruiter uploads resumes, the backend instantly confirms the upload and begins processing behind the scenes. Because of this, the Streamlit interface never freezes — recruiters can continue working while the AI processes resumes, and the dashboard updates automatically once ranked results are ready.

Technology Stack

Layer	Technology	Purpose
Backend API	FastAPI	Serves the application logic and exposes endpoints for uploads, scoring, and results
Frontend UI	Streamlit	Recruiter-facing interface for uploads and the ranked results dashboard
Relational Database	Neon PostgreSQL	Stores structured data: jobs, candidates, resumes, and final scores
Vector Database	Qdrant Cloud	Stores resume embeddings and powers semantic candidate search
Caching & Rate Limiting	Upstash Redis	Caches AI responses and protects the API from overload
Extraction & Explanation Model	Groq LLM (Llama-3)	Extracts structured data from resumes and writes score explanations
Embedding Model	Google Gemini	Converts resume and job text into vector embeddings for semantic search

AI / ML Workflow

Every resume moves through a four-stage AI pipeline:

1. Parsing — raw text is extracted from each uploaded PDF or DOCX file.
2. Extraction — the Groq Llama-3 model reads the raw text and converts it into a structured profile: skills, years of experience, education, and past roles.
3. Vector Embedding — the structured profile is converted into a numeric vector using Google Gemini. This vector captures the meaning of the resume, not just its words.
4. Semantic Search — the vector is stored in Qdrant. When a job description is embedded the same way, the platform searches for the candidate vectors that are closest in meaning to the job requirements.

Implementation

The platform is built entirely in asynchronous Python. Async and await are used for every I/O-heavy operation, such as calling AI models, querying databases, and reading files. This lets the backend handle many resumes and many recruiters at the same time, without one slow task blocking another.

The codebase is organized into clean, separate layers: API routes, core configuration, business logic services, database models, and validation schemas. This separation keeps the code easy to read, easy to test, and easy to extend as the platform grows.

Resume Processing

AI models cannot always be trusted to return perfectly formatted data on their own. To solve this, the platform uses Pydantic, a Python data-validation library, to define the exact shape that extracted resume data must follow — for example, the candidate's name, skill list, years of experience, and education history.

When the AI model returns a resume profile, the response is checked against this Pydantic schema. If the output is incomplete or incorrectly formatted, the system catches the error before it ever reaches the database. This "forced JSON" approach makes resume extraction reliable and consistent, even across thousands of different resume formats and writing styles.

AI / RAG Pipeline

The platform's semantic search does not simply look for exact keyword matches. Instead, it converts both the resume and the job description into embeddings — numeric vectors that represent the meaning of the text.

Texts with similar meaning produce vectors that sit close together, even when the exact wording is different. For example, "led a team of developers" and "managed a software engineering team" use different words but describe the same experience, so their vectors end up close together in the system. Qdrant, a database built specifically for vector search, quickly finds the resumes whose meaning is closest to the job's requirements. This retrieval-based approach is what produces the AI Semantic Score.

Backend & Database

PostgreSQL, hosted on Neon, acts as the system of record. It stores job postings, candidate profiles, resume metadata, and final scores in a structured and reliable format that is easy to query for reports and dashboards.

Redis, hosted on Upstash, plays two supporting roles. First, it protects the API through rate limiting, preventing the system from being overwhelmed by too many requests at once. Second, it caches AI responses, so if the same resume or a similar request is processed again, the platform can reuse a recent result instead of calling the AI model a second time. This makes the platform both faster and cheaper to run.

Results

The platform delivers three consistent outcomes:

- Fast — asynchronous background tasks and response caching allow the system to process batches of resumes without ever freezing the user interface.
- Explainable — every ranked candidate comes with a short, plain-language explanation, so recruiters understand exactly why a score was given instead of trusting a hidden number.
- Accurate — combining semantic understanding with rule-based checks produces rankings that are more reliable than either method used alone.

Testing / Evaluation

The platform was evaluated using a labeled test set, comparing the system's candidate ranking against an expected ranking for each job description. This testing shows high precision in identifying genuinely qualified candidates.

The hybrid algorithm is a key reason for this accuracy. It acts as a safety net against AI hallucination. If the semantic model ever misjudges the meaning of a resume, the rule-based checks — exact skill overlap and years of experience — keep the final score grounded in real, verifiable evidence found in the resume text. Neither part of the score is allowed to work alone.

Screenshots

The platform's dashboard uses a clean, dark-mode interface, designed for recruiters who spend long sessions reviewing candidates. The left panel holds the job description and resume upload controls. The main panel displays the ranked candidate list, with each candidate shown as a card containing their name, an overall match score, a small breakdown of the semantic and rule-based scores, a set of highlighted matching skills, and the AI-generated explanation paragraph underneath. The dark background and clear score badges keep the most important information easy to scan at a glance.

Deployment

The entire platform is containerized using Docker, which keeps the development and production environments identical. The FastAPI backend is deployed on Render, and the Streamlit frontend is deployed on Streamlit

Cloud. All three databases — Neon PostgreSQL, Qdrant Cloud, and Upstash Redis — are fully managed, serverless services. This means there are no servers to patch or scale manually, and the platform can grow to handle more traffic without any infrastructure changes.

Challenges

- Forcing structured JSON from the LLM — language models sometimes add extra text or return slightly malformed data. This was solved with strict Pydantic validation, careful prompt design, and automatic retries when a response failed validation.
- Cloud database connection drops — serverless, free-tier databases can go idle and drop connections after inactivity. This was handled with connection pooling and automatic retry logic in the backend.
- API rate limits — third-party AI services enforce rate limits on how many requests can be made. This was managed using Redis-based request throttling and response caching, which reduces the number of repeated AI calls.

Design Decisions

Why Hybrid Scoring Is Safer Than Pure AI Scoring

A pure AI semantic score, on its own, can be influenced by how a resume is phrased rather than what a candidate has actually done. This creates a risk that a resume which simply "sounds" like a good fit could be ranked highly without the underlying facts to support it.

Adding a rule-based layer — checking exact skill overlap and confirmed years of experience — grounds every score in verifiable facts taken directly from the resume. This combination gives recruiters a ranking they can trust and explain, and it significantly lowers the risk of an unqualified candidate rising to the top simply because of clever wording.

Future Improvements

- Email notifications — automatically alert recruiters by email once a batch of resumes has finished ranking.
- OCR for scanned resumes — add optical character recognition so image-based or scanned resumes can be read and processed as well.
- Admin analytics dashboard — give hiring managers visibility into hiring trends, pipeline volume, and time-to-shortlist across all job openings.

Conclusion

The AI Resume Screening Platform shows how modern AI tools can be applied responsibly to a real hiring problem. By combining structured extraction, semantic search, and rule-based checks, the platform turns a slow, error-prone manual process into a fast, transparent, and explainable one. Its asynchronous, cloud-native architecture keeps it responsive and scalable, while the hybrid scoring approach keeps every result grounded in real evidence rather than guesswork. Together, these design choices make the platform a practical foundation for building fair and efficient recruitment tools.